



Lecture 12 — Data Cleaning using Pandas

BTDS331 — Introduction to Data Science | Module 2: Data Collection and Preprocessing

CO: CO2 | **Bloom's Level:** K2 (Understand)

This notebook is a **teaching companion** for Lecture 12. For every Pandas method used in the lecture slides, you will find three things:

1. **What it is** — a one-line definition
2. **Why we use it** — the purpose / real-world reason
3. **Code** — a runnable example on our own messy practice dataset

◇ Use this notebook to explain each concept live to students, then let them run the cells themselves and predict the output before executing.

1. What is Data Cleaning?

Definition: Data Cleaning is the process of identifying and correcting inaccurate, incomplete, duplicate, inconsistent, or improperly formatted data to improve data quality before analysis.

Goal: Convert ◇ Raw Data → ◇ High-Quality Data

The Data Cleaning Process (pipeline):

Raw Data (errors, missing values, duplicates) → Cleaning (identify & fix, standardize) → Reliable Data (clean, consistent, accurate, complete) → Analysis (better insights & decisions)

Why is Data Cleaning Important?

- **Improves Accuracy** — accurate data gives accurate insights
- **Removes Errors** — fixes mistakes and inconsistencies
- **Reduces Noise** — keeps only relevant and useful data

- **Better Machine Learning Models** — clean data improves model performance
- **Reliable Business Decisions** — strong data leads to confident decisions
- **Saves Time during Analysis** — clean data is easy to work with

Famous Principle

Garbage In → Garbage Out (GIGO) — if the input data is bad, the output will be bad, no matter how good your analysis technique is.

Common Data Quality Problems

Problem	Example
Missing Values	Age is blank
Duplicate Records	Same student entered twice
Incorrect Entries	Name typed as <code>three</code> instead of <code>3</code>
Typographical Errors	<code>'Delhi</code> with a stray apostrophe
Invalid Data	Age = <code>-8</code> or <code>250</code>
Inconsistent Formatting	<code>delhi</code> vs <code>Delhi</code> vs <code>DELHI</code>
Extra Spaces	<code>" Pune "</code>
Wrong Data Types	Salary stored as text <code>"fifty five thousand"</code>
Outliers	Salary = 1,000,000 when everyone else earns ~50,000

2. Introduction to Pandas

What is Pandas? Pandas is an open-source Python library used for data manipulation, cleaning, analysis, and preprocessing. Developed by **Wes McKinney (2008)**.

Why Pandas?

- **Fast** — high performance for data analysis
- **Easy to Use** — simple, intuitive syntax
- **Works with Large Datasets** efficiently
- **Powerful Data Cleaning Functions**

Installing & Importing Pandas

```
pip install pandas
```

```
In [1]: # Import pandas – 'pd' is the worldwide standard alias
import pandas as pd

print("Pandas version:", pd.__version__)
```

Pandas version: 3.0.5

3. Understanding the DataFrame

A **DataFrame** is a two-dimensional table consisting of **rows** and **columns** — think of it like an Excel spreadsheet living inside Python. Each column can hold a different data type.

A single column of a DataFrame is called a **Series** (a Series is 1-dimensional; a DataFrame is a collection of Series sharing the same index).

```
In [2]: # A DataFrame built manually from a dictionary (rows = students, columns
sample = pd.DataFrame({
    "Roll": [101, 102, 103],
    "Name": ["Rahul", "Aman", "Priya"],
    "Marks": [89, 75, 92]
})
sample
```

```
Out[2]:
```

	Roll	Name	Marks
0	101	Rahul	89
1	102	Aman	75
2	103	Priya	92

4. Reading a CSV File — `pd.read_csv()`

What it is: A function that loads data from a `.csv` file into a Pandas DataFrame.

Why we use it: Almost all real-world data cleaning starts by loading data from a file (CSV, Excel, database export, etc.). `read_csv()` is the most common entry point.

```
df = pd.read_csv("students.csv")
```

◇ Note: Make sure the CSV file is in the same directory as your notebook, or provide the full/correct file path.

We will use our own **messy practice dataset**:

`students_data_cleaning_practice.csv` (12 student records containing missing values, a duplicate row, and invalid ages/marks — exactly the kind of problems described in the lecture).

In [9]: `pip install openpyxl`

```

Downloading openpyxl-3.1.5-py2.py3-none-any.whl.metadata (2.5 kB)
Collecting et_xmlfile (from openpyxl)
  Downloading et_xmlfile-2.0.0-py3-none-any.whl.metadata (2.7 kB)
Downloading openpyxl-3.1.5-py2.py3-none-any.whl (250 kB)
Downloading et_xmlfile-2.0.0-py3-none-any.whl (18 kB)
Installing collected packages: et_xmlfile, openpyxl

```

[illegible]

[illegible]

[illegible]

Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5
Note: you may need to restart the kernel to use updated packages.

```
In [10... import openpyxl
```

```
In [11... # Load the messy practice dataset
df = pd.read_excel(r"C:\Users\DELL\Downloads\Students_Data_Cleaning_Pract
df
```

```
Out[11]:
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Bengaluru	88.0
1	102	Priya	19	Mysuru	91.0
2	103	Rahul	19	Bengaluru	88.0
3	103	Rahul	19	Bengaluru	88.0
4	104	Aman	-5	Delhi	75.0
5	105	Neha	200	Mumbai	96.0
6	106	Kiran	21	NaN	82.0
7	107	Anu	22	Chennai	NaN
8	108	Ravi	20	Bengaluru	88.0
9	109	Meena	19	Mumbai	105.0
10	110	Divya	20	Hyderabad	79.0
11	111	Arun	21	NaN	67.0

5. `df.head()` — Preview the Top Rows

What it is: Returns the **first 5 rows** of the DataFrame by default (you can pass a number, e.g. `df.head(10)`, to see more/fewer rows).

Why we use it: To quickly glance at the data right after loading it — check column names, data format, and get a first impression of data quality — without printing the whole dataset.

```
In [4]: df.head()      # first 5 rows
```

```
Out[4]:
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Bengaluru	88.0
1	102	Priya	19	Mysuru	91.0
2	103	Rahul	19	Bengaluru	88.0
3	103	Rahul	19	Bengaluru	88.0
4	104	Aman	-5	Delhi	75.0

```
In [5]: df.head(3)     # you can pass a custom number of rows
```



```
Out[5]:
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Bengaluru	88.0
1	102	Priya	19	Mysuru	91.0
2	103	Rahul	19	Bengaluru	88.0

6. `df.tail()` — Preview the Bottom Rows

What it is: Returns the **last 5 rows** of the DataFrame by default.

Why we use it: To check the end of the dataset — useful for spotting issues that appear later in the file (e.g. footer/summary rows accidentally included, or records appended at a different time with a different format).

```
In [6]: df.tail()      # last 5 rows
```

```
Out[6]:
```

	ID	Name	Age	City	Marks
7	107	Anu	22	Chennai	NaN
8	108	Ravi	20	Bengaluru	88.0
9	109	Meena	19	Mumbai	105.0
10	110	Divya	20	Hyderabad	79.0
11	111	Arun	21	NaN	67.0

7. `df.shape` — Dataset Dimensions

What it is: An attribute (not a function — no parentheses!) that returns a tuple: `(number_of_rows, number_of_columns)`.

Why we use it: To instantly know the size of the dataset — how many records and how many fields we are working with. This is often the very first thing you check.

```
In [12... df.shape
```

```
Out[12]: (12, 5)
```

```
In [8]: rows, cols = df.shape
print(f"Rows: {rows}")
print(f"Columns: {cols}")
```

Rows: 12
Columns: 5

8. `df.columns` — Column Names

What it is: Returns all the column names of the DataFrame as an `Index` object (convert with `.tolist()` to get a plain Python list).

Why we use it: To know exactly what fields exist in the dataset — necessary before you can select, rename, or clean any specific column.

```
In [9]: df.columns
```

```
Out[9]: Index(['ID', 'Name', 'Age', 'City', 'Marks'], dtype='str')
```

```
In [10]: df.columns.tolist() # as a plain Python list
```

```
Out[10]: ['ID', 'Name', 'Age', 'City', 'Marks']
```

9. `df.info()` — Structure Summary

What it is: Displays a concise summary of the DataFrame: column names, the count of **non-null** values per column, and the **data type (dtype)** of each column.

Why we use it: This is one of the most important first checks in data cleaning — it immediately tells you the structure of the dataset **and** which columns have missing values (compare the "Non-Null Count" to the total number of rows).

```
In [11]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 5 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0    ID      12 non-null    int64  
 1   Name     12 non-null    str     
 2    Age     12 non-null    int64  
 3   City     10 non-null    str     
 4   Marks    11 non-null    float64 
dtypes: float64(1), int64(2), str(2)
memory usage: 612.0 bytes
```

Reading the output above:

- `RangeIndex: 12 entries` → total number of rows
- `City` and `Marks` show a **Non-Null Count lower than 12** → those columns have missing values
- `dtypes` tells us whether pandas is treating a column as text (`object`), whole numbers (`int64`), or decimals (`float64`)

10. `df.describe()` — Statistical Summary

What it is: Generates descriptive statistics (count, mean, std, min, 25%, 50%, 75%, max) for all **numeric** columns.

Why we use it: To spot **invalid values and outliers** at a glance. For example, if `min` Age is negative or `max` Age is 200, that's an immediate red flag — a real age can never be negative or above ~120.

```
In [13... df.describe()
```

```
Out[13]:
```

	ID	Age	Marks
count	12.000000	12.000000	11.000000
mean	105.750000	32.916667	86.090909
std	3.278719	53.114727	10.241626
min	101.000000	-5.000000	67.000000
25%	103.000000	19.000000	80.500000
50%	105.500000	20.000000	88.000000
75%	108.250000	21.000000	89.500000
max	111.000000	200.000000	105.000000

Try it yourself: Look at the `Age` and `Marks` rows above.

- Does `min` Age make sense?
- Does `max` Age make sense?
- Does `max` Marks make sense (marks should be 0–100)?

```
In [13... # describe(include='all') also summarizes text/object columns (Name, City)
df.describe(include='all')
```

```
Out[13]:
```

	ID	Name	Age	City	Marks
count	12.000000	12	12.000000	10	11.000000
unique	NaN	10	NaN	6	NaN
top	NaN	Rahul	NaN	Bengaluru	NaN
freq	NaN	3	NaN	4	NaN
mean	105.750000	NaN	32.916667	NaN	86.090909
std	3.278719	NaN	53.114727	NaN	10.241626
min	101.000000	NaN	-5.000000	NaN	67.000000
25%	103.000000	NaN	19.000000	NaN	80.500000
50%	105.500000	NaN	20.000000	NaN	88.000000
75%	108.250000	NaN	21.000000	NaN	89.500000
max	111.000000	NaN	200.000000	NaN	105.000000

11. `df.dtypes` — Checking Data Types

What it is: An attribute that returns the data type of every column (`int64` , `float64` , `object` for text, etc.).

Why we use it: Machine Learning algorithms and mathematical operations require the **correct** data type. If a numeric column like Salary is accidentally stored as text (e.g. `"fifty five thousand"`), pandas will read it as `object` , and you won't be able to do arithmetic on it until it's fixed.

```
In [14... df.dtypes
```

```
Out[14]: ID          int64
         Name        str
         Age         int64
         City        str
         Marks       float64
         dtype: object
```

12. Checking Missing Values — `df.isnull()` and `df.isnull().sum()`

What it is:

- `df.isnull()` returns a DataFrame of the **same shape**, with `True`

where a value is missing and `False` where a value is present.

- `df.isnull().sum()` adds those `True / False` values up **column-by-column** (`True` counts as 1), giving you the **count of missing values per column**.

Why we use it: To find out exactly *which* columns have missing data and *how many* values are missing, so you can decide how to handle them (drop, fill with mean/median/mode, etc.).

```
In [15... df.isnull() # True = missing, False = present (whole grid)
```

```
Out[15]:
```

	ID	Name	Age	City	Marks
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
5	False	False	False	False	False
6	False	False	False	True	False
7	False	False	False	False	True
8	False	False	False	False	False
9	False	False	False	False	False
10	False	False	False	False	False
11	False	False	False	True	False

```
In [16... df.isnull().sum() # missing value count PER COLUMN – the most commonly
```

```
Out[16]: ID      0
          Name    0
          Age     0
          City    2
          Marks   1
          dtype: int64
```

Interpretation: A `0` next to a column means that column has **no missing values**. Any number greater than `0` tells you exactly how many records need attention in that column.

13. Detecting Duplicate Records — `df.duplicated()`

What it is: Returns `True` for every row that is an **exact duplicate** of a previous row (by default, Pandas compares **all columns** to decide if a row is a duplicate).

Why we use it: Duplicate records inflate counts, distort averages/statistics, and can seriously bias analysis and ML models if not removed.

```
In [17]: df.duplicated()           # True marks a row that repeats an earlier row
```

```
Out[17]: 0    False
          1    False
          2    False
          3     True
          4    False
          5    False
          6    False
          7    False
          8    False
          9    False
         10    False
         11    False
          dtype: bool
```

```
In [18]: df.duplicated().sum()     # total count of duplicate rows in the dataset
```

```
Out[18]: np.int64(1)
```

```
In [19]: # See the actual duplicate row(s)
         df[df.duplicated()]
```

```
Out[19]:
```

	ID	Name	Age	City	Marks
3	103	Rahul	19	Bengaluru	88.0

14. Removing Duplicates — `df.drop_duplicates()`

What it is: Returns a **new** DataFrame with duplicate rows removed (the original `df` is left unchanged unless you pass `inplace=True` or reassign it).

Why we use it: Once duplicates are detected, this is the standard way to

remove them and keep only one copy of each unique record.

```
df_clean = df.drop_duplicates()           # returns a new
DataFrame
df.drop_duplicates(inplace=True)          # OR modifies df
directly
```

```
In [20... df_no_duplicates = df.drop_duplicates()
print("Shape before:", df.shape)
print("Shape after: ", df_no_duplicates.shape)
df_no_duplicates
```

Shape before: (12, 5)

Shape after: (11, 5)

```
Out[20]:
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Bengaluru	88.0
1	102	Priya	19	Mysuru	91.0
2	103	Rahul	19	Bengaluru	88.0
4	104	Aman	-5	Delhi	75.0
5	105	Neha	200	Mumbai	96.0
6	106	Kiran	21	NaN	82.0
7	107	Anu	22	Chennai	NaN
8	108	Ravi	20	Bengaluru	88.0
9	109	Meena	19	Mumbai	105.0
10	110	Divya	20	Hyderabad	79.0
11	111	Arun	21	NaN	67.0

15. Identifying Invalid Values (Logical Validation)

What it is: Checking values against real-world logical rules — not every error shows up as a missing value or a duplicate. A number can be **present, correctly typed, and still wrong** (e.g. Age = 200).

Why we use it: Some errors are only caught through domain knowledge / business rules, e.g.:

- Age should logically be between **0 and 120**
- Salary cannot be **negative**
- Marks cannot **exceed 100**

`df.describe()` (Section 10) is a great first pass for spotting these via min/max — but we can also filter explicitly to see the offending rows.

```
In [21... # Rows with an illogical Age (negative, or above a realistic human age li
invalid_age = df[(df["Age"] < 0) | (df["Age"] > 120)]
invalid_age
```

```
Out[21]:
```

	ID	Name	Age	City	Marks
4	104	Aman	-5	Delhi	75.0
5	105	Neha	200	Mumbai	96.0

```
In [22... # Rows with an illogical Marks value (assuming marks are out of 100)
invalid_marks = df[(df["Marks"] < 0) | (df["Marks"] > 100)]
invalid_marks
```

```
Out[22]:
```

	ID	Name	Age	City	Marks
9	109	Meena	19	Mumbai	105.0

16. Putting It All Together — The Cleaning Workflow

The lecture summarizes the process as a pipeline:

```
Collect Dataset → Load into Pandas → Inspect Dataset →
Find Missing Values
    → Detect Duplicates → Check Data Types → Identify
Invalid Data
    → Clean Dataset → Ready for Analysis
```

Let's run the full inspection workflow on our dataset in one place — this is exactly what you would type at the very start of any real data cleaning project.

```
In [23... # STEP 1-2: Load into Pandas
df = pd.read_csv("students_data_cleaning_practice.csv")

# STEP 3: Inspect Dataset
print("First 5 rows:")
display(df.head())

print("\nShape (rows, columns):", df.shape)

print("\nColumn names:", df.columns.tolist())
```



```
print("\nInfo:")
df.info()
```

First 5 rows:

	ID	Name	Age	City	Marks
0	101	Rahul	20	Bengaluru	88.0
1	102	Priya	19	Mysuru	91.0
2	103	Rahul	19	Bengaluru	88.0
3	103	Rahul	19	Bengaluru	88.0
4	104	Aman	-5	Delhi	75.0

Shape (rows, columns): (12, 5)

Column names: ['ID', 'Name', 'Age', 'City', 'Marks']

Info:
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 5 columns):
Column Non-Null Count Dtype
--- ---
0 ID 12 non-null int64
1 Name 12 non-null str
2 Age 12 non-null int64
3 City 10 non-null str
4 Marks 11 non-null float64
dtypes: float64(1), int64(2), str(2)
memory usage: 612.0 bytes

```
In [24... # STEP 4: Find Missing Values
print("Missing values per column:")
print(df.isnull().sum())
```

Missing values per column:
ID 0
Name 0
Age 0
City 2
Marks 1
dtype: int64

```
In [25... # STEP 5: Detect Duplicates
print("Number of duplicate rows:", df.duplicated().sum())
display(df[df.duplicated()])
```

Number of duplicate rows: 1

	ID	Name	Age	City	Marks
3	103	Rahul	19	Bengaluru	88.0

```
In [26... # STEP 6: Check Data Types
```

```
print(df.dtypes)
```

```
ID          int64
Name        str
Age         int64
City        str
Marks       float64
dtype: object
```

```
In [27... # STEP 7: Identify Invalid Data (logical validation)
print("Invalid ages (outside 0-120):")
display(df[(df["Age"] < 0) | (df["Age"] > 120)])

print("Invalid marks (outside 0-100):")
display(df[(df["Marks"] < 0) | (df["Marks"] > 100)])
```

Invalid ages (outside 0-120):

	ID	Name	Age	City	Marks
4	104	Aman	-5	Delhi	75.0
5	105	Neha	200	Mumbai	96.0

Invalid marks (outside 0-100):

	ID	Name	Age	City	Marks
9	109	Meena	19	Mumbai	105.0

```
In [28... # STEP 8: Clean the Dataset (example fixes – combine everything learned a
df_clean = df.copy()

# a) remove exact duplicate rows
df_clean = df_clean.drop_duplicates()

# b) fill missing City with 'Unknown' and missing Marks with the column m
df_clean["City"] = df_clean["City"].fillna("Unknown")
df_clean["Marks"] = df_clean["Marks"].fillna(df_clean["Marks"].mean())

# c) fix invalid ages/marks by clipping to a realistic valid range
df_clean["Age"] = df_clean["Age"].clip(lower=0, upper=120)
df_clean["Marks"] = df_clean["Marks"].clip(lower=0, upper=100)

df_clean
```

```
Out[28]:
```

	ID	Name	Age	City	Marks
0	101	Rahul	20	Bengaluru	88.0
1	102	Priya	19	Mysuru	91.0
2	103	Rahul	19	Bengaluru	88.0
4	104	Aman	0	Delhi	75.0
5	105	Neha	120	Mumbai	96.0
6	106	Kiran	21	Unknown	82.0
7	107	Anu	22	Chennai	85.9
8	108	Ravi	20	Bengaluru	88.0
9	109	Meena	19	Mumbai	100.0
10	110	Divya	20	Hyderabad	79.0
11	111	Arun	21	Unknown	67.0

```
In [29... # STEP 9: Ready for Analysis – verify the dataset is now clean
print("Missing values left:", df_clean.isnull().sum().sum())
print("Duplicate rows left:", df_clean.duplicated().sum())
print("Any invalid ages left:", ((df_clean['Age'] < 0) | (df_clean['Age']
print("Any invalid marks left:", ((df_clean['Marks'] < 0) | (df_clean['Ma

Missing values left: 0
Duplicate rows left: 0
Any invalid ages left: 0
Any invalid marks left: 0
```

17. Quick Reference Table

Method	What it does	When to use
<code>pd.read_csv()</code>	Loads a CSV file into a DataFrame	Step 1 — always
<code>df.head()</code>	Shows first 5 rows	Quick preview
<code>df.tail()</code>	Shows last 5 rows	Check end of data
<code>df.shape</code>	(rows, columns)	Know dataset size
<code>df.columns</code>	List of column names	Know available fields
<code>df.info()</code>	Structure + non-null counts + dtypes	Overview + spot missing data
<code>df.describe()</code>	Statistical summary of numeric columns	Spot outliers/invalid values
<code>df.dtypes</code>	Data type per column	Verify correct types
<code>df.isnull()</code>	True/False grid of missing values	Detailed missing-value

Method	What it does	When to use
		map
<code>df.isnull().sum()</code>	Missing value count per column	Quantify missing data
<code>df.duplicated()</code>	True/False flag per row	Detect duplicate rows
<code>df.duplicated().sum()</code>	Total duplicate count	Quantify duplicates
<code>df.drop_duplicates()</code>	Removes duplicate rows	Clean duplicates

18. Practice Questions (from the lecture)

Try answering these — then verify using the code cells above:

1. Define Data Cleaning.
2. Why is Data Cleaning necessary before analysis?
3. What is Pandas?
4. Differentiate a Series and a DataFrame.
5. Explain the purpose of `head()`, `tail()`, `shape`, and `info()`.
6. Write Python code to read a CSV file using Pandas.
7. How do you identify missing values in a dataset?
8. How can duplicate records be detected using Pandas?

Mini Activity

Download any dataset from [Kaggle](#) and repeat every step in this notebook on it: `head()`, `tail()`, `shape`, `columns`, `describe(include='all')`, `isnull().sum()`, `duplicated().sum()`.

Next Lecture: Missing Values and Outliers (handling strategies — drop, fill, impute)